



Go to WGU

r/WGU

•

23d ago

Vuxerius

B.S. IT--Security

D385 V2 (Software Security and Testing) - (Really) Rough Guide for Tasks 1 & 2

Information Technology

I just got the official pass for my first attempt at D385 V2 (2 Tasks) after about 1 full day of working on it. And since I've mostly been lurking, I wanted to contribute and help guide anyone who needs it.

For context, the only experience I have with Python was the OA (now PA) version of Python for IT Automation (D522), which I took just before this course.

Additionally, some of the instructions can be really confusing and contradictory (at least to me), which ended up being the bulk of the difficulty I faced during this course. From my understanding, the V2 of this course was cut down at the last minute, and the instructions in the rubric, task files, and resources don't really match up at points.

This guide is going to be just a general outline of what to do and what you need to submit. As I'm not sure what information I can share exactly, ***I kept all the info within the scope of the general objectives and what students were given as a template.*** If you need any assistance, let me know, and I can clarify some of my poorly written sentences. I tried to organize it as best as I could, but I can be terrible at explaining things, so please bear with me.

After skimming the course material, I decided to skip it and just delve into the tasks, as it is comprised of documentation links and textbook sections. I tried using the resources on WGU Connect, and they weren't very helpful either.

If you haven't used Visual Studio Code before or need help just getting the tasks ready to work on, the Additional Resources Version 2 section of the Course Connect has an intro video. Other than that, the rest of the resources are essentially just links to Python documentation.

General Notes/Tips

- Do not try coding and running these tasks from the ZIP file directly. You will run into errors. Make sure to extract them into a separate folder, I called mine 'wguCoding' to make things easier to ZIP and submit.

- Both tasks will ask for a specific number of screenshots for certain parts. I would follow that amount exactly.
- The sample code and test_app files contain a bunch of comments that guide or even answer problems for you.
- The comments can also be deceitful and can contain references to objectives that no longer exist. Try not to get tripped up by these comments.
- Sometimes the objective will ask you to describe things in a general sense, not necessarily things you have done in the application.
- If you're like me, and probably messed up installing something and the Pytest command they gave you isn't working in the terminal, try `'python -m'` + the original run command.
- Remember to save the code and test cases before you run your tests
- You **DO NOT** have to solve all of the 'TODO:' sections in the source code for **Task 2. The in-code comments saying you have to do ALL the TODO items is a lie. Just complete/solve as many problems as each section of the rubric calls for.**
- Your security mitigation report is just your Word document.
- You do not have to actually save the file as app_solution.py
- For Task 2, fix up the API Endpoint section a bit to ensure your tests for your solutions run properly.
- Task 2 has TODO comments by some code problems, which tell you what they can be used for once solved, i.e., one problem once solved can be used for section A and section D.
- Task 1 wants you to submit the ZIP of the application files and report with screenshots in it **separately**. (Two uploads: ZIP of application files, DOCX report)
- Task 2 wants you to submit a **single** ZIP upload with ALL the files. (PDF with screenshots, Report DOCX, and the edited application files.) Screenshots are in a separate PDF as per that weird note on all of the tasks (*"All supporting documentation, such as screenshots and proof of experience, should be collected in a pdf file and submitted separately from the main file"*)
- Pytest flags '-s' and '-rP' for Task 2 are good to know.
- **SAVE ALL CHANGES BEFORE YOU ZIP EVERYTHING TO SUBMIT**

Coding Notes/Tips

- A bit exaggerated, but almost 90% of solutions for both task vulnerabilities will involve assert and try/except code. [Helpful guide on both](#)
- Task 1 involves a ton of logging, so be sure to get comfortable with creating and using the [logger](#) throughout the application
- A lot of the solutions involve working with user-inputted strings. W3 Schools has a list of functions you can use with string objects to enforce input validation: [String Methods](#)
- A lot of the code in Task 1 does not need to be removed. A large portion of Task 1 is simply adding input validation to ensure that the already built-in logic does not break. [Input Validation Code ABC's Resource Linked from Connect](#)
- For Task 1, you need to know the different types of [built-in exceptions](#). There are at least three you need to know for the try/except block
- For creating your user-friendly error messages, refer to [message flashing with Flask](#)

- Making tests in the test_run file is essentially assert statements for almost every single one.
- Knowing how to use [environment variables](#) is highly recommended.
- Task 2 imports SQLite3. Depending on what vulnerabilities you want to solve, I recommend learning the bare basics at a minimum of how databases look in Python and how to utilize them for the API Endpoints section and testing.
- The API key is located in the auth header. The expected format of the header is "Bearer <key>" (As indicated in the source code comments guiding you).

Task 1

Section A is very straightforward and is made up mostly of writing a report. For A1 and A2, you need to describe what static code analysis is and then provide 5 issues that it detects. A3 is essentially looking into the code itself and pointing out 5 distinct issues and explaining why they are bad. I personally just combed through the code and pointed out things I found wrong, but you can use the static analysis report file, which lists the line numbers for issues.

I'm not sure if this is needed, but I put the line numbers and the lines of code where I found issues inside my report just to make sure I got everything covered.

Section B is where the coding happens. Luckily, the starter program they give you has lots of helpful comments, usually starting with 'TODO:'. Some of these quite literally give you the answer.

B1. You need to set up a logger that creates a file called "Troubleshooting_studentID.log" (Your student ID goes where 'studentID' is). For the configuration of the logger, the starter template has a TODO section with how they want the logger to be set up. I used those instructions in conjunction with [Python's documentation](#) to create my logger.

B2 and B3 are fairly easy, as the comments in the source code point to where you need to implement both the try/except block and the assertion statements. I referred to W3 Schools to get the gist of [try/except blocks](#) and [assert statements](#). The comments also guide you on what the conditions should be for your defensive coding, such as making sure inputs are a string and not empty. You also want to make sure that you are using the logger you created in B1 under the assert statements and inside the try/except blocks. For the message the logger outputs, I just followed whatever the print statement I replaced said, or if there wasn't an existing one, I made a generic message that described what was being logged.

B4. This is probably the simplest part of the section, though I saw some confusion from other students on what exactly needed to be captured. What I did was just a screenshot of Visual Studio open, with the application (it's a locally run website you open in a browser) in a small window on top of VS. That way, I could capture both the console & code from VS *and* the browser window of the application up and running.

B5. You need to create test cases for the application to pass. **Ignore the comment about section A7.** There are 2 test cases already made for you, so you can use those as guides for making the others.

In total, you just need to show **4 different** screenshots that 4 test cases have passed in your report. The console command to run these test cases is at the top of the file in the guiding comments. I'm almost certain you will get this returned if you submit 1 screenshot of all 4 test cases. To play it extra safe, I took a screenshot of each individual line in the console that said (testcase) PASSED and paired it with a screenshot of the test case code. Might be overkill, but it felt really odd just submitting one slim screenshot of (testcase) PASSED.

C1a. I just summarized what log entries are generally used for, then gave some examples of how we used them in the application.

C1b is specifically about the application we updated. For this, you need to run the application and literally try to mess it up as much as possible. Submit negative values where it doesn't make sense, input incorrect values and cause errors, etc., etc. This will generate abnormalities (if you implemented logging properly) in the log file for you to use in C1b. Pick two of the abnormalities found in the log file, describe them, and then explain what could be inferred. To clear this section up, I listed the abnormality, copied the full line from the log file, and then explained what it was and what it indicates as far as security is concerned.

C2a. Just describe what you implemented into the program. I didn't implement any breakpoints, so I just talked about logging and pytest.

C2b. Refer to your screenshots/test cases from B5; these specifically have to be issues you tested for in B5. Luckily, these line up generally with the vulnerabilities you found in A3 and what you tested for in B5, so it should be easy to explain what defect you found, and what you did to fix it. Make sure to describe each defect individually. I just made 4 small paragraphs.

C3 is kind of a repeat of other sections. Just explain **specifically, input validation, exception handling, and assertion statements** that you used in your code, and how they solved vulnerabilities/issues in the code.

C4 is asking for a generic summary of what static code can detect, but you can tie it back into your application if you want to strengthen the report.

Once you are done, upload the report and ZIP as two separate files in the portal. **(THIS CHANGES FOR TASK 2 I DON'T KNOW WHY)** For the ZIP file, I just included everything that came with the original task files, which is more than what the task is asking for (Python app and log file).

Task 2

Personally, this is where things got insanely confusing instruction-wise for certain sections. I had to go back and forth between my code and the report many times, as I missed certain things or the report wanted something that the coding instructions did not mandate.

For this task, you need to submit 3 things total in one ZIP file: A PDF of the screenshots you take for Sections A-C5 (i.e Word document exported as PDF), your application files (I just left it as a folder in the final ZIP file), and your written report (DOCX)

Section A: Follow this exactly, or it will make the entire section harder

You can look inside the Flake8 report to find vulnerabilities you want to solve, or just follow the commented sections to find each vulnerability. **You do not need to solve all of them, just 2.** For section A, ensure that they are labelled 'GENERAL SECURITY VULNERABILITY' as the API ones are for section B. I highly recommend doing vulnerability 1 and 2 for both sections A and B, as section C becomes easier. **If you pick vulnerabilities 1 and 2 for the API section, make sure you edit the functions under 'API Endpoints', or some of your test cases WILL NOT WORK. You should also tackle the exception handling issues in that section so you don't have to go back later.**

Before you solve the vulnerabilities, take a screenshot of the existing code. **(One screenshot for each problem you'll be solving)** Make sure to include the line numbers. You can either just stick them immediately into the Word document that you are going to export as a PDF, or you can put them in a folder to combine into a PDF later.

Do not delete the insecure code; use # to comment it out. This note is at the end of section A, and if you start coding before you finish reading section A, you might have already deleted the old code.

Once you comment out the insecure code, start coding your fix for the vulnerability. Then take a screenshot of your solutions. **(One screenshot per solution, 2 total)**

Then take **ONE** screenshot that shows the old code commented out. For this, I took a screenshot of the bottom half of problem 1 and the top half of problem 2 within the Visual Studio viewport. Making sure to get every line of the commented-out old code in frame.

Section B: Follow the exact same instructions as Section A, but with problems labelled 'API SECURITY VULNERABILITY'

Section C:

This is where your fixes will be tested. You need to first import the app into the test_run file, along with whatever you need to test the problems you solved. i.e., importing the user database from the application. Then, uncomment the test fixture that is provided to you.

You need to create at least 4 test cases. 1 for each vulnerability you solved.

Fortunately, the file basically has the answers for how you need to create your tests, if you picked general vulnerability 1 and 2, as well as API vulnerability 1 and 2 to solve.

Depending on how you coded the solutions for the vulnerabilities, the provided test cases may work simply by uncommenting them, or you may have to use them as a reference when making your own.

Once you are done making the tests, run them and then do the following.

C1. Screenshot one photo of the Visual Studio window, including the console with the tests being run and the code of some of the tests in the coding viewport. This ensures you show that the tests are using the proper naming conventions ("test_" prefix). I think it also covers that the application is using the pytest framework, but it was worded oddly to me, so I don't know if they mean just simply that the test file exists, or that it was run against the application we updated. For the sake of ensuring I covered everything, I just showed the tests being run against the application.

C2. I just took one screenshot of a few of my tests that contained assert statements. I made sure that the screenshot had at least 2 assert statements, as I'm not sure how pedantic evaluators are when it comes to the rubric stating assert statements specifically in a plural sense.

C3. This one sounded really similar to C1, but the keyword here is that the tests have to pass. For this, I just took a screenshot of the console with the command I used and the 4 tests being passed.

C4. To get this, you might have to run extra flags with the test command, especially if you used print commands for the pass message, like the examples showed.

You can use '-s' so even failed tests have the print statement, or '-rP' to show print statement messages only if the test passed. I personally used '-rP'. Once you see your tests with the print statements you created in the console/terminal, take a screenshot of them.

C5. This was another one that seemed really redundant, but what I did was show each individual descriptive comment and passed test (in console), along with the corresponding test code in the viewport. I did this for every single test to get the four screenshots.

Section D:

This is where you write the report. The instructions here started to get really confusing when I was doing the task, so I will explain in detail what I did in case you have the same issue. The guide at the VERY BOTTOM of the app_student source code can help.

D1. I picked two mitigation strategies, each one solving 2 of the issues I provided fixes for in the prior sections. Essentially, come up with a mitigation strategy that covers 2 issues you fixed in the prior sections. Then come up with another one that covers the remaining 2.

To really clarify how I did mine:

First Mitigation Strategy -> Covers the Two Section A Issues

Second Mitigation Strategy -> Covers the Two Section B Issues

This is the opposite of what the guidance says at the bottom of the source code. They outline one vulnerability being handled by 2 mitigation strategies. So 4 vulnerabilities total, 8 total mitigations.

I honestly just went off the rubric wording. TWO mitigation strategies to remediate the selected FOUR vulnerabilities. My logic was that so long as the two mitigation strategies covered the four vulnerabilities, it would be fine. So honestly, I'm not sure if this is supposed to be 2 strategies that apply to all 4 vulnerabilities, coming up with 1 strategy for the two pairs of vulnerabilities, or the two strategies per vulnerability.

D2. The wording on this one is kind of odd. But to keep it simple, I just came up with two examples of input validation that I implemented in the fixes I coded. This is because whatever you select could be from the application or a potential future project. Personally, it is way easier to defend and back up the usage of input validation from what work I did for the coding problems than to come up with a hypothetical. However, as the wording implies, both are acceptable.

D3. Choose the fixes you made in the API endpoints section for this. There are specific parts in the endpoint section where you can implement the required fixes for this section. Originally, I had not done this and had to go back to fix up the API endpoint section to be able to have something to answer for this part. The Section D written report guidance at the bottom of the page gives you examples of what you can fix and then talk about. **You don't need screenshots of the fixes for D3 unless you picked whatever solution you did for sections A or B.** You just need to code them and describe them in the report.

As for writing, I just referenced what I edited/fixed, and why the change is better than the original code.

D4. This is the one that confused me the most. As the entire assignment has essentially been about our application. This section **IS NOT** asking about our application. Write a general explanation about an encryption method used to secure application traffic.

This encryption method DOES NOT have to be used within your fixes or the application. It's simply asking for a description and explanation of one. This one part had me confused for a few hours, as the rubric states, "*used to secure **application traffic** in the **provided script** or report.*" This is odd because inside the starter code, the example methods they give you, 2/3 of them are not for application traffic.

I don't know if I was just being dumb that night, but I figured out after that they meant you need to explain one of the methods used for securing application traffic, which are FOUND listed as comments in the provided script or report. Not one that is specifically utilized in the script.

It has also been confirmed on Connect that, despite being in the source code guide for D4, you are not to use password hashing as a method.

Once you're done, ZIP up the files and submit the task.

Conclusion/Asides

Apologies that the guide is super long (and poorly organized), I just wanted to cover as much as I could to hopefully make this course as painless as possible. The course material is quite dry, and the resources on Connect didn't really help much (could change in the future). This is my first time making a guide, as I didn't see too much on this version of the course yet (fairly new as of writing), and I hoped to create the guide I would've wanted to see when going through the tasks.

Overall, this course was fairly easy with my limited coding experience, as it mainly consisted of assert statements and conditional logic (if that's even a correct term). I'm honestly just glad I didn't get to experience the OA version of this course.